

Łódź University of Technology
International Faculty of Engineering
Individual Plan of Study
Citation-Grounded Retrieval-Augmented Generation
pipeline for Densely Cross-Referenced Corpora

Volodymyr Hryshko

Supervised by: dr Bartosz Sakowicz

September 21, 2026

Abstract

This report documents the design and implementation of a retrieval-augmented generation (RAG) system that answers questions grounded in uploaded documents and returns source-cited responses. The system targets densely cross-referenced documents—legislation, regulatory policies and standards—whose meaning depends on their internal structure and on the provisions they cite. It is a Python pipeline served over HTTP to a desktop interface, and it runs entirely on local hardware. Ingestion recovers each document’s hierarchy down to paragraphs, points and subparagraphs, together with the page numbers printed on its pages, and stores every passage with that provenance. Queries that name a location—an article, a paragraph, a page—are routed to deterministic metadata lookup; all other queries go to a hybrid of dense semantic search and BM25 lexical scoring, fused by reciprocal rank fusion. The system, not the user, decides how many passages a question receives. A locally hosted language model answers from those passages only and cites them by number. Part I describes ingestion, retrieval, generation and the interface. Part II describes what it took to make the system output cross-references: parsing structure below the level of a provision, extracting references into a graph, and presenting them—including why an interactive graph visualisation was replaced by a static reference tree. The report closes with the current status and limitations of the system.

Contents

1	Overview	4
1.1	Aim	4
1.2	Requirements	4
1.3	Architecture	5
1.4	Development timeline	5
1.5	Structure of this report	5
I	Grounded Question Answering	8
2	Ingestion	8
2.1	Reading documents	8
2.2	Cleaning	8
2.3	Structural units	9
2.4	Chunking	10
2.5	Printed page numbers	10
2.6	Embedding and storage	11
3	Retrieval	11
3.1	Hybrid search	11
3.2	Exact lookup for named locations	12
3.3	Deciding how many passages	13
4	Generation	14
4.1	Model and runtime	14
4.2	Prompt	15
4.3	Deterministic replies	16
4.4	Response	16
5	Interface	17
5.1	HTTP API	17
5.2	Desktop application	17
II	Cross-Referencing	19
6	Structure below the provision	19
6.1	Paragraphs, points and sub-points	19
6.2	Text that closes a list	19
6.3	Asking for a subdivision	20
6.4	Writing references for the reader	21

7	Cross-reference graph	21
7.1	Extracting references	21
7.2	Building edges	21
7.3	Storage	22
7.4	Neighbourhood of an answer	23
7.5	Presenting cross-references	23
	Status and Outlook	24
8	Status and limitations	24
8.1	Testing and continuous integration	24
8.2	Measured behaviour	24
8.3	Limitations	24
8.4	Future work	25
8.5	Summary	25
	References	27

1 Overview

1.1 Aim

The aim of the project is to implement the Retrieval-Augmented Generation (RAG) technique [1] by building a Python-driven pipeline that answers questions grounded in uploaded documents, giving back accurate, source-cited responses.

A RAG system answers in two steps. It first retrieves the passages of a document collection that bear on a question, then asks a language model to answer from those passages alone. Grounding the answer in retrieved text ties it to the documents the user supplied rather than to whatever the model absorbed during training, and it makes every answer checkable: each claim can be traced to a passage the reader can open.

The system is aimed at densely cross-referenced documents such as legislation, regulatory policies and standards. The EU Artificial Intelligence Act [7] and the Canadian Tri-Council Policy Statement on research ethics [8] served as the main development corpora, alongside other regulations. Documents of this kind share two properties that a generic RAG pipeline handles poorly.

- **Questions name locations.** A reader asks about “paragraph 2 of Article 7” or “chapter 3, page 48” rather than describing content. To a similarity search, “Article 7” is two tokens: it retrieves passages that *mention* Article 7, which are rarely Article 7 itself.
- **Meaning is distributed.** A provision often states only part of a rule and delegates the rest: “a measure requested under Article 93”, “the condition under paragraph 1, point (b)”. An answer taken from one passage in isolation can be literally correct and still misleading.

1.2 Requirements

Three requirements follow from the aim and from these properties.

R1 Grounding. Answers are drawn only from passages of the uploaded documents. When the passages do not contain the answer, the system says so instead of answering from general knowledge.

R2 Citation. Every passage carries its provenance: the document, the page number a reader sees printed on the page, and the passage’s position in the document’s hierarchy, written the way a reader would cite it (“Article 101(1), second subparagraph”).

R3 Structure. A question that names a part of a document retrieves exactly that part, and the provisions it cites can be followed from the answer.

Two constraints shaped the implementation. First, the system runs entirely on local, consumer-grade hardware—the development machine has an NVIDIA GTX 1650 with 4 GB of video memory and 16 GB of RAM—so no document leaves the machine and no query incurs a fee. Second, nothing may be specific to one legal system: structural parsing and query handling are written against general numbering conventions, not against the vocabulary of one document.

1.3 Architecture

Figure 1 shows the two paths through the system. *Ingestion* turns an uploaded file into labelled passages and a graph of cross-references; *answering* turns a question into retrieved passages and a generated answer. Both run in a Python backend (**backend/**) exposed as an HTTP API, and a desktop application (**frontend/**) presents the results. Table 1 maps the backend packages to their responsibilities and Table 2 lists the technologies used.

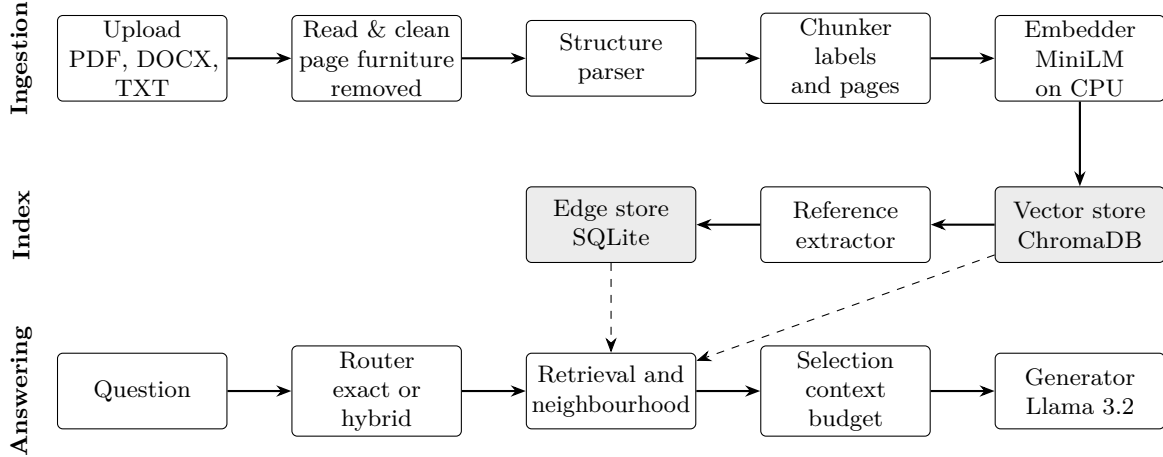


Figure 1: The ingestion and answering paths. Shaded nodes are persistent stores; dashed arrows are reads from them during answering. The reference extractor reads the stored passages back in reading order, so the edges it produces use exactly the labels the passages carry.

Package	Responsibility
ingestion	Reading PDF, DOCX and TXT files; cleaning; structural parsing; chunking; printed page numbers; reference extraction
embedding	Sentence embeddings and the ChromaDB vector store, including metadata lookups
retrieval	Query parsing, routing to exact lookup, hybrid dense and lexical search, passage selection
generation	The generator interface and its Ollama-backed implementation
graph	Cross-reference edges, their SQLite store and neighbourhood queries
pipeline	The two entry points, ingestion and answering; context budget; deterministic replies
api.py	FastAPI endpoints
telemetry.py	CPU, GPU and memory readings shown in the interface

Table 1: Backend packages.

1.4 Development timeline

Table 3 summarises how the system was built. Development proceeded from the bottom of the pipeline upwards: ingestion first, then retrieval and generation, then structure and cross-references, with the interface growing alongside.

1.5 Structure of this report

Part I covers the system up to the cross-reference graph: ingestion (Section 2), retrieval (Section 3), generation (Section 4) and the interface (Section 5). Part II covers what it took to make the

Layer	Technology	Reason for the choice
Pipeline	Python 3.12	The reference ecosystem for text processing, embeddings and model runtimes.
API	FastAPI, Uvicorn	Typed request models; the interface and the tests call the same endpoints.
PDF reading	PyMuPDF, pypdf	PyMuPDF exposes layout geometry—line positions and font sizes—which footer removal and page numbering rely on; pypdf is the fallback.
DOCX reading	python-docx	Paragraph-level access to Word documents.
Chunking	LangChain text splitter	Recursive splitting on paragraph, line, sentence and word boundaries.
Embeddings	all-MiniLM-L6-v2	384-dimensional sentence embeddings [4, 5], fast enough on a CPU.
Vector store	ChromaDB	Embedded and persistent; filters on metadata inside the store.
Lexical scoring	rank-bm25	Okapi BM25 [2] without an external search engine.
Language model	Llama 3.2 (3B), Ollama	Runs locally within 4 GB of video memory.
Graph store	SQLite	Part of the Python standard library; no server to run.
Desktop shell	Tauri 2	Uses the system web view, so the application stays small.
Interface	React 19, TypeScript	Typed components, built with Vite and styled with Tailwind CSS.
Telemetry	psutil, NVML	CPU, memory and GPU readings.

Table 2: Technology stack.

system output cross-references: parsing structure below the level of a provision (Section 6) and the graph built on it, including how it is presented (Section 7). Section 8 closes with the current status, measured behaviour and limitations.

Period	Milestones
April–May 2026	Ingestion layer: readers for PDF, DOCX and TXT behind one loader; fixed-size chunking; unit tests on a PDF fixture. Local embedding layer on ChromaDB.
June 2026	Generator interface and the first end-to-end pipeline, retrieving by embedding similarity alone. Repository split into backend/ and frontend/ . Endpoints for health, upload and query; chat pane wired to the backend; text cleaning; generation through Ollama.
June–July 2026	Queries scoped to one document; token usage shown per answer; hybrid retrieval with BM25 and reciprocal rank fusion; retrieval returns chunk identifiers and metadata.
August 2026	Structural markers and page boundaries kept during ingestion; queries naming a provision routed to exact metadata lookup; live CPU and GPU telemetry; passage locations shown in the interface.
Early September 2026	Passages ordered against the lost-in-the-middle effect; embedder moved to the CPU; symbol-font glyphs stripped; past answers replayable; per-document statistics; structural units detected generically; answers split into a direct answer and a supporting detail.
Mid September 2026	Cross-reference extraction and edge building; paragraph and point labels; running headers and footers removed; edges stored and a neighbourhood returned with each answer; continuous integration; an interactive cross-reference graph; desktop window sizing and native translucency.
Late September 2026	Graph view replaced by a reference tree; retrieval of the exact subdivision named, with typo tolerance; commands and query cancellation; printed page numbers; passage count decided by the system; text that closes a list recognised as a subparagraph; plain-language summaries for bare references.

Table 3: Development timeline.

Part I

Grounded Question Answering

2 Ingestion

Ingestion turns an uploaded file into passages that carry their provenance. It runs once per upload, in the order shown in Listing 1: the file is read page by page, cleaned, split into labelled chunks, given the page numbers printed on its pages, embedded and stored, and finally scanned for cross-references (Part II).

2.1 Reading documents

A single loader dispatches on the file extension to a reader for PDF, DOCX or plain text; the interface checks the extension before uploading, and the loader rejects any other type. The loader returns a list of (page number, text) pairs, so that page boundaries survive into the chunks. Only PDF files have real pages; a Word or text file is returned as a single page.

PDF files are read with PyMuPDF, which reports every line of a page together with its bounding box and font size. This geometry matters because of two kinds of page furniture. Footnotes and running footers sit visually below the body text, but once the layout is flattened into a text stream they land in the middle of it: a footnote ends up spliced into the very clause that cites it, where it corrupts both the passage and any reference found in it. The reader therefore walks each page's lines from the bottom up and drops the trailing run of lines that are either smaller than the body text or lie in the bottom 5 % of the page, stopping at the first line of body text. The body size is taken to be the font size that carries the most characters on the page. A recital marker such as “(148)” is also set small, but it always has body text below it, so the walk never reaches it. If PyMuPDF fails, the reader falls back to pypdf, which extracts text without geometry.

2.2 Cleaning

Extracted text is normalised before it is parsed:

- Unicode is normalised to NFC, and typographic dashes, quotes, bullets and ellipses are

```
1 def ingest(doc_path: str) -> int:
2     pages = load_document_pages(doc_path)
3     chunks = with_printed_pages(chunk_document(pages),
4                                 load_printed_pages(doc_path))
5     doc_name = os.path.basename(doc_path)
6
7     embed_and_store_chunks(chunks, doc_name)
8     build_graph(doc_name)
9
10    return len(chunks)
```

Listing 1: The ingestion entry point (pipeline/pipeline.py).

mapped to plain equivalents.

- Box-drawing and geometric symbols are removed, together with code points from the Unicode Private Use Area. Symbol fonts such as Wingdings place their bullets there, and extraction copies them as raw code points that render as empty boxes.
- Control characters and lines consisting only of a number are removed, runs of spaces are collapsed, and three or more blank lines become two.
- Text that lost its spaces during extraction—detected when fewer than 8% of its characters are spaces—has letter runs of twelve or more characters split into words with a word-frequency model (wordninja).

Running headers and footers that survive the geometric filter are removed across pages. The first and last four lines of every page are reduced to a *signature* in which every run of digits becomes #, so that “Page 12 of 144” and “Page 13 of 144” share one signature. A signature that appears on at least $\max(3, \lfloor P/2 \rfloor)$ of the document’s P pages is page furniture and is removed everywhere before the pages are joined. The threshold is deliberately conservative: a genuine line repeated on half of all pages is rare, whereas a running header that survives turns up in the middle of passages and in retrieval results.

2.3 Structural units

Densely cross-referenced documents announce their structure in their text: headings such as “Article 7”, “Chapter III” or “Annex VIII”, recital numbers such as “(148)”, and numbered paragraphs and lettered points. The structure parser finds these markers and the chunker turns them into chunk boundaries and labels. This section describes the units down to the level of a provision; the subdivisions inside a provision are the subject of Section 6.

A heading is a unit word—*Article*, *Section*, *Clause*, *Rule*, *Annex*, *Schedule*, *Exhibit*, *Appendix*, *Chapter*, *Part* or *Title*—followed by a number in any of the forms the development corpora use: 58, 3.13, 3.7A, VII or B. It must stand alone on its line or be followed by a capitalised title, which rules out sentences that merely begin with a reference. Article numbers must also form a sequence: the first may be at most 3, and each later one must be greater than the one before. A heading-like line that breaks the sequence—typically a reference to another article that happens to begin a line—is rejected. Recitals are numbers in parentheses alone on their line and are accepted only in the order 1, 2, 3, . . . , which rejects footnote markers of the same shape.

Each unit has a depth (Table 4). The parser keeps the current path from the outermost unit to the innermost, and a marker takes over its own depth and closes everything deeper: a new article closes the previous article’s paragraph and point, but not its chapter. Every chunk is labelled with the full path it falls under, such as *chapter III*, *article 7*, *paragraph 2*, *point b*.

None of the unit words is specific to one legal system, and none is required. A document in which fewer than three markers are found is treated as unstructured and chunked plainly; retrieval then relies on similarity alone.

Depth	Units
0	annex, schedule, appendix, exhibit
1	part
2	title
3	chapter
4	section
5	article, recital, clause, rule
6	paragraph
7	point, subparagraph
8	sub-point

Table 4: Structural depths. A marker closes every open unit at its own depth or deeper.

2.4 Chunking

The cleaned pages are joined into one string, and the offset at which each page starts is recorded, so that any position in the text can be mapped back to its page by binary search. The text between two consecutive markers is split with a recursive character splitter into chunks of at most 500 characters with an overlap of 50, preferring to break at blank lines, then line ends, then sentence ends, then spaces. Each chunk is stored with its text, the pages on which it starts and ends, and the structural labels of its path.

Trade-offs. Small chunks make retrieval precise: one point of a list is usually one chunk, so a citation points at the sentence that matters rather than at a page of text. The cost is that a long provision spans many chunks. This is acceptable because a named provision is never retrieved by similarity: it is reassembled from its labels in reading order (Section 3). Cutting at markers before splitting by size also means that no chunk straddles two provisions, so every label a chunk carries is exact.

2.5 Printed page numbers

The page number a reader sees is often not the PDF’s own page index. Front matter pushes the printed numbering behind: the page of TCPS 2 that prints “48” is the 56th page of the PDF. A reader who asks about “page 48” means the printed number, and a citation should show it. The printed number sits in the footer, which the reader has just removed, so it is recovered from the page geometry instead.

For every page p , the numbers standing at the start or end of a line in the top and bottom 8 % of the page are collected, including the first number of forms such as “117/144” and “L 176/5”. Each such number n proposes an offset $p - n$. Let G_p be the set of offsets proposed on page p . The document’s offset is the one proposed by the most pages,

$$o^* = \arg \max_o |\{p : o \in G_p\}|, \quad (1)$$

and it is accepted only if at least $\max(3, P/2)$ of the P pages agree on it. Every page p with $p - o^* \geq 1$ is then given the printed number $p - o^*$, and every chunk receives the printed numbers of the pages on which it starts and ends. For TCPS 2 the offset is 8; for the AI Act, whose footers read “117/144”, it is 0.

The agreement threshold is what keeps this general. Numbers in the margins that are not page numbers—dates, article numbers, journal references—propose offsets that vary from page to page and never accumulate. A document numbered per section (“3.8-2”) or not at all never reaches agreement and keeps the PDF numbering, which is the correct fallback. The method assumes a single dominant numbering run: front matter numbered in roman numerals simply receives no printed number, and a document whose numbering restarts several times is numbered by its longest run.

2.6 Embedding and storage

Chunks are embedded with all-MiniLM-L6-v2 [5], a sentence-transformer [4] that maps text to a 384-dimensional vector, and stored in a persistent ChromaDB collection that uses cosine distance. Each chunk’s identifier is `{document}_chunk_{i}`, so reading order can be recovered by sorting identifiers. Its metadata holds the source document, the start and end pages (PDF and printed) and the structural labels. Plain numbers are stored as integers and all other identifiers as text (58 but “3.7A”), because the store’s equality filters are typed. Uploading a document again first deletes its previous chunks, and writes are batched below the store’s maximum batch size, which large regulations exceed.

Trade-offs. The embedder runs on the CPU. On a 4 GB GPU every megabyte of video memory taken by the embedder is a megabyte the language model cannot use, and the model already has to offload part of its layers to system memory. Embedding a large document on the CPU takes longer, but it happens once per upload, and a query needs only one short text embedded. ChromaDB was chosen over a bare vector index such as FAISS because it evaluates metadata filters—equality, conjunction, disjunction and numeric ranges—inside the store, which exact lookups (Section 3) depend on. Its filters offer no substring or pattern matching, which is why the parsing effort goes into producing exact labels at ingestion rather than into matching loosely at query time.

3 Retrieval

Retrieval decides which passages the language model sees, and therefore bounds what any answer can say. It has two paths. A question that names a location in a document is answered by *exact lookup* on the stored labels; any other question goes to *hybrid search*, which combines semantic and lexical similarity. The router tries exact lookup first and falls back to hybrid search when the question names nothing or nothing carries the named labels. Besides the passages, it returns a note stating whether the lookup was exact, which subdivision the question asked for, whether that subdivision was found, and, if not, what the provision contains instead (Section 6).

3.1 Hybrid search

Dense ranking. The question is embedded with the same model as the passages, and every chunk in scope—one selected document or the whole collection—is ranked by cosine distance to it.

```

1 def _rrf(rank_list: list[list[str]], k: int = 60) -> dict:
2     scores: dict[str, float] = {}
3     for ranking in rank_list:
4         for rank, _id in enumerate(ranking):
5             scores[_id] = scores.get(_id, 0.0) + 1.0 / (k + rank)
6     return scores

```

Listing 2: Reciprocal rank fusion (`retrieval/hybrid.py`).

Lexical ranking. The same chunks are ranked by Okapi BM25 [2] over lower-cased word tokens.

Fusion. The two rankings are combined by reciprocal rank fusion [3] (Listing 2). Each chunk d scores

$$\text{RRF}(d) = \sum_{r \in \{\text{dense}, \text{BM25}\}} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60, \quad (2)$$

where $\text{rank}_r(d)$ is the chunk’s position in ranking r , counted from 0.

Trade-offs. The two rankings fail in complementary ways. Dense embeddings capture paraphrase—“who can complain” finds a provision on the right to lodge a complaint—but blur exact tokens: numbers, amounts and defined terms (“Article 92”, “EUR 15 000 000”) contribute little to a sentence vector. BM25 matches exact tokens but knows nothing of synonyms. Fusing by rank rather than by score avoids having to reconcile two incomparable scales, a bounded cosine distance and an unbounded BM25 score, and needs no tuned weights. The cost is that magnitudes are discarded: a passage far ahead in one ranking gets no more credit than one narrowly ahead. The BM25 index is rebuilt over the chunks in scope for each question. Over the 2019 chunks of the AI Act, hybrid retrieval takes about 235 ms in total, which is small beside generation; a collection orders of magnitude larger would need a persistent inverted index instead.

3.2 Exact lookup for named locations

Parsing the question. A pattern finds unit words followed by a number (“article 66”, “Art. 66”, “articles no. 5”, “page 48”, “section 3.13”). Plain numbers become integers and anything else upper-cased text, matching how the labels were stored (3.7a matches 3.7A). Some conventions name a unit by another word, so a unit is also tried under an alternative reading: in EU acts, “clause 148” means recital 148, and “section 5” may mean article 5. A question that matches nothing is checked for a mistyped unit word: any word of four to ten letters followed by a number is compared with the unit vocabulary, and a similarity ratio of at least 0.75 accepts it, so “artilce 66” is read as article 66.

Building filters. Each named unit becomes one or more filters on the stored metadata. A page becomes a range condition, because a chunk may run over two pages: first on the printed page numbers, then on the PDF’s own page numbers (Listing 3). When a question names several units, all of them are combined first, so that “chapter 3 page 48” retrieves the passages on

```

1 def _alternatives(field, value):
2     if field != "page":
3         return [{field: {"$eq": value}}]
4     if not isinstance(value, int):
5         return []
6
7     return [
8         {"$and": [{"printed_page": {"$lte": value}},
9                  {"printed_page_end": {"$gte": value}}]},
10        {"$and": [{"page": {"$lte": value}},
11                 {"page_end": {"$gte": value}}]},
12        {"page": {"$eq": value}},
13    ]

```

Listing 3: Filters for one named unit; a page matches every chunk that runs over it (retrieval/hybrid.py).

printed page 48 *within* chapter 3. Only if no chunk carries them all is each unit tried on its own, the most specific first: page, then recital, clause, rule, article, section, annex, schedule, chapter and part. Trying the units separately from the outset returns chapter 3’s opening passages for that question, because “chapter 3” alone matches first; the combination is what lets a page be addressed inside a chapter.

Returning the provision. The first filter that matches returns every chunk of the provision, up to 200, in reading order—not the k most similar chunks. A named provision is the answer to “what does it say” in full, and similarity has no role in deciding which of its parts matter. The subdivisions inside it are handled in Section 6.

Trade-offs. Exact lookup is deterministic and explainable: the same question always returns the same passages, and the interface can say “exact match” instead of presenting a similarity score as if it measured correctness. It is also fast, taking 3–6 ms against roughly 235 ms for hybrid search. Its weakness is that it depends on the question naming units in a form the parser recognises. A form it does not recognise falls through to hybrid search, so the worst case is the behaviour of an ordinary RAG system, never an empty answer.

3.3 Deciding how many passages

Early versions let the user choose how many passages to retrieve, between 2 and 12. A fixed number fits no question well: it truncates a long provision that was named in full, and it pads a focused question with passages that merely share its vocabulary. The number is now decided by the system, from what the question asked for:

- **A named subdivision** (“paragraph 2 of Article 7”): exactly the chunks that carry the requested labels, which for a paragraph includes all of its points and sub-points (Section 6).
- **A named provision** (“Article 93”): the whole provision in reading order.

```

1 def _close_to_best(chunks, distances, metas):
2     if not distances:
3         return chunks, distances, metas
4
5     best = 1 - min(distances)
6     keep = []
7     for i, distance in enumerate(distances):
8         if i < _FEWEST or 1 - distance >= best - _MARGIN:
9             keep.append(i)
10    keep = keep[:_MOST]
11
12    return ([chunks[i] for i in keep], [distances[i] for i in keep],
13            [metas[i] for i in keep])

```

Listing 4: Keeping the passages close to the best one (`retrieval/hybrid.py`).

- **Any other question:** the candidates that score close to the best one. Of the twelve best candidates after fusion, candidate i (counted from 0) is kept if

$$i < 3 \quad \text{or} \quad s_i \geq \max_j s_j - 0.15, \quad (3)$$

where s_i is its cosine similarity to the question, and at most eight are kept (Listing 4).

On every path the passages are then admitted in order while their total length stays within 2 000 words. At about 1.4 tokens per word this is roughly 2 800 tokens, which leaves room in the model’s 4 096-token context window for the instructions and the answer.

Trade-offs. The margin is relative rather than absolute because similarities between a question and the passages of one document cluster tightly: for typical questions on the AI Act the top twelve candidates fell between 0.57 and 0.86, so an absolute threshold would keep everything for one question and nothing for the next. The minimum of three protects against a lone passage cut mid-sentence, and the maximum of eight against a question so generic that everything scores alike. The margin and bounds were set by inspecting score distributions on the development corpora rather than learned, and they fix only the upper limit; the context budget decides the rest.

4 Generation

4.1 Model and runtime

Answers are generated by Llama 3.2 with 3 billion parameters, served locally by Ollama with a 4 096-token context window, a temperature of 0 and JSON output mode. At temperature 0 the same passages and question produce the same answer, which makes behaviour reproducible and differences between versions of the pipeline attributable to the pipeline. On the development machine about four fifths of the model’s layers fit in the 4 GB of video memory and the rest run from system memory, so generation speed is bounded by memory bandwidth rather than by the GPU: an answer takes 7–9 s once the model is loaded, and about 20 s when it has to be loaded

```

1 [The question names Article 101(1). The passages below are that
2 text; answer from them.]
3 You answer questions about a document using ONLY the numbered
4 context passages below.
5 Do two things:
6 1. ANSWER: answer the question directly and briefly.
7 2. DETAIL: say where in the document the answer comes from and what
8     the surrounding text adds, citing passages by their number,
9     e.g. [2]. If the document only names something without
10    describing it, say so plainly instead of repeating the name.
11 Do not add anything the passages do not say.
12 If the passages do not contain the answer, say that they do not.
13 Respond with a JSON object EXACTLY in this shape:
14 { "answer": "...", "detail": "..."}
15
16 Context:
17 [1] ...
18 [3] ...
19 [2] ...
20
21 Question: ...

```

Listing 5: The prompt, abridged (`generation/basic_backend.py`).

first. The interface warns in advance when the model is not loaded.

The generator sits behind an abstract interface with a single method, `generate(query, chunks, focus)`, so a different model or runtime can be substituted by adding one class without touching retrieval or the API.

Trade-offs. A local 3B model keeps every document on the machine, costs nothing per question and works offline. The price is weaker synthesis than a large hosted model and a small context window. Both push work into retrieval: the passages must already be the right ones, in the right order, and within budget, because the model cannot compensate for a poor selection.

4.2 Prompt

The prompt (Listing 5) states the model’s role, gives it two tasks, tells it what to do when the passages do not contain the answer, and fixes the shape of its output. The *answer* responds to the question directly; the *detail* states where in the document the answer comes from, citing passages by number, and says plainly when the document only names something without describing it.

Passage order. Language models attend most reliably to the start and end of a long context and least to its middle [6]. The passages arrive ranked, so they are interleaved before being placed in the prompt: the first, third, fifth and so on from the front, then the rest in reverse, which puts the strongest passages at both ends (for six passages the order is 1, 3, 5, 6, 4, 2). Each passage keeps the number it had before reordering, and that number is the one the interface shows, so “[3]” in the detail is Source 3 in the citations pane.

Focus. When exact lookup found the subdivision the question named, the prompt opens with a line naming it: “The question names Article 101(1). The passages below are that text; answer from them.” Without it, a small model treats the passages as loose matches and may answer from a neighbouring passage that reads as relevant.

4.3 Deterministic replies

Three situations are answered without relying on the model’s judgement.

A named part that does not exist. A question may name a subdivision the provision does not have, such as paragraph 2 of an article that is a single paragraph. Given the rest of the article, a small model answers anyway, presenting a neighbouring sentence as the requested paragraph. The model is therefore not called. The reply is composed from the stored labels, and the whole provision is shown in the citations pane:

Article 85 has no paragraph 2: its text is not divided into paragraphs.

Article 7 has no paragraph 9. Its paragraphs are 1, 2, 3.

A bare reference. A query such as “paragraph 1 article 101” names a location but asks nothing, and with nothing to answer the model tends to return an empty answer. A query is treated as a bare reference when nothing but filler words (“show”, “the text of”) remains once every recognised reference is removed from it. The question given to the model is then replaced by a task that makes the chat reply worth having next to the passages themselves: “Explain in plain language what Article 101(1) says.”

A non-answer. If the model’s answer is empty or a placeholder (“None”, “null”, “N/A”, “unknown”), it is replaced by a notice that the passages are in the citations pane, rather than displayed as if it were an answer.

4.4 Response

Every answer is returned as one JSON object: the answer and its detail; the passages; for each passage, its document, pages and structural labels; the cross-reference neighbourhood of the passages (Section 7); the lookup note; retrieval and generation times; and the prompt and completion token counts reported by Ollama together with the size of the context window.

No relevance percentage. Earlier versions showed a relevance percentage for each passage, derived from its cosine similarity to the question. It was removed because it measured how closely the wording matched, not whether the answer was right, and because passages found by exact lookup always showed 100 %. The lookup note replaced it: the interface states whether the passages are an exact match on the named provision or the closest passages by keyword and meaning, which is what the reader needs in order to judge them.

5 Interface

5.1 HTTP API

The backend is served by FastAPI (Table 5). Answering and retrieval take the same request, a query and an optional document to scope it to. The number of passages is deliberately not a parameter: it is decided by the retrieval (Section 3).

Method	Path	Purpose
GET	/health	Liveness check.
GET	/stats	CPU, GPU and memory load and temperatures.
GET	/documents	Indexed documents and their chunk counts.
POST	/upload	Store and ingest a file.
DELETE	/document/{name}	Remove a document’s chunks and cross-reference edges.
GET	/document/{name}/stats	Pages and structural units found.
POST	/query	Retrieval-augmented answer.
POST	/retrieve	Passages only, without generation.
GET	/document/{name}/provision	Full text of one provision, in reading order.
GET	/document/{name}/neighbours	What a provision cites and what cites it.

Table 5: API endpoints.

5.2 Desktop application

The interface is a React 19 application written in TypeScript and built with Vite, running inside a Tauri 2 desktop shell. It is divided into three resizable panes that can each be folded away (Figure 2).

Screenshot of the application: save it as `images/interface.png`.

Figure 2: The desktop application: retrieved citations (left), chat (centre) and context (right).

Retrieved citations. Each passage is shown as a card headed by its source number, its printed page and its reference written the way it would be cited—for example “p. 118 · Article 101(1), second subparagraph”. A line above the cards states how the passages were found: an exact match on the provision named, or the closest passages by keyword and meaning. Below each card, the provisions the passage cites can be followed (Section 7).

Chat. The transcript shows the question, the answer, and the detail beneath it in smaller type. Every answered turn keeps its own passages, so clicking an earlier answer restores its citations without querying again. A running query can be stopped. The composer shows which document questions are scoped to and accepts a small set of commands, suggested as the first word is typed: **find**: retrieves passages without generating an answer, **doc**: changes the scope, **clear** empties the conversation and **help** lists the commands. Shorthand for structural words is expanded with the Tab key: **a**, **para**, **p**, **sec** and **cha** become *article*, *paragraph*, *point*, *section* and *chapter*. Only Tab expands, so the word “a” in an ordinary sentence is left alone.

Context. The last query’s token usage is shown against the 4096-token window, split into prompt and response, together with the session total and the time taken. Machine telemetry shows GPU and CPU load, temperature and memory, and how much of the model is resident in video memory. The document list shows each indexed document with a structural summary (“144 pages · 13 chapters · 113 articles · 180 recitals”), accepts uploads of PDF, DOCX and TXT files with an estimate of the indexing time, and scopes questions to a document when it is clicked.

Trade-offs. Tauri was chosen over Electron because it uses the operating system’s web view instead of bundling a browser, which keeps the application small and its memory footprint low beside a language model competing for the same machine. The cost is that three web engines render the interface—WebKit on macOS and Linux, WebView2 on Windows—and small differences between them must be accommodated. Native translucency is used only where the platform composites it itself (vibrancy on macOS, Mica and Acrylic on Windows); elsewhere the window stays opaque, because a blur redrawn by the web view on every change of the panes made the interface sluggish.

Part II

Cross-Referencing

6 Structure below the provision

Cross-references in dense documents rarely point at a whole provision. They point inside one: “Article 66(2)(e)(iv)”, “paragraph 1, point (b)”, “the second subparagraph of paragraph 3”. Readers ask questions the same way. To resolve either, every passage has to know not only its article but its paragraph, point, sub-point and subparagraph. This was the first thing it took to make the system output cross-references: until passages carry these labels, a reference can be resolved only to a whole provision, and “Article 66(2)(e)” cannot be told apart from Article 66.

6.1 Paragraphs, points and sub-points

Inside a provision three kinds of marker open a line: a numbered paragraph (“2.”), a lettered point (“(b)” or “b.”) and a roman sub-point (“(iv)” or “iv.”). The shapes alone are ambiguous: “(i)”, “(v)” and “(x)” are both letters and roman numerals, and a number followed by a full stop can begin a wrapped line in the middle of a sentence.

The parser therefore resolves markers by *sequence*. After each heading it expects paragraph 1 and point (a); it then tracks the next paragraph number, the next point letter and, once a point is open, the next roman numeral. A marker is accepted only if it is the one expected next (Listing 6). After point (h), “(i)” is the expected letter and becomes point (i). After point (k) of Article 7(2) of the AI Act—“the extent to which existing Union law provides for:”—the expected letter is (l), so the “(i)” that follows is taken as the first sub-point of (k). A paragraph number is accepted only if it is the next one expected and the text before it ends a sentence or a clause (with “.”, “;” or “:”), which rejects numbers that merely begin a line.

6.2 Text that closes a list

A paragraph that introduces a list of points often continues after the list with text that applies to the whole paragraph, not to its last point. EU drafting calls it a *subparagraph*. Article 101(1)

```
1 elif kind == "marker" and next_paragraph is not None:
2     if value == next_point:
3         marks.append((pos, "point", value))
4         next_point = chr(ord(value) + 1)
5         next_subpoint = "i"
6     elif value == next_subpoint:
7         marks.append((pos, "subpoint", value))
8         index = _ROMANS.index(value)
9         if index + 1 < len(_ROMANS):
10             next_subpoint = _ROMANS[index + 1]
11         else:
12             next_subpoint = None
```

Listing 6: Accepting a point or sub-point only in sequence (`ingestion/structure.py`, abridged).

p.117	Article 101	Article 101 Fines for providers of ...
p.117	Article 101(1)	1. The Commission may impose ... fines
p.117	Article 101(1)(a)	(a) infringed the relevant provisions ...
p.117	Article 101(1)(b)	(b) failed to comply with a request ...
p.117	Article 101(1)(c)	(c) failed to comply with a measure ...
p.118	Article 101(1)(d)	(d) failed to make available ...
p.118	Article 101(1), 2nd sub.	In fixing the amount of the fine ...
p.118	Article 101(2)	2. Before adopting the decision ...

Listing 7: Labels assigned to the chunks of Article 101 of the AI Act.

of the AI Act lists four grounds for a fine in points (a) to (d) and then states, without a marker of its own, how the amount of the fine is fixed. Nothing in the text marks where point (d) ends, so a parser that only reads markers would give the fining criteria the label of point (d)—and a question about point (d) would receive them too.

The end of a list is recognised from the punctuation of formal drafting. Items before the last end with a semicolon, a comma or a conjunction; only the last ends a sentence. So after the last item of a list, a full stop followed by a line that begins with a capital letter starts the closing text. Inside a numbered paragraph the closing text is labelled as the paragraph’s second subparagraph (third, and so on, after further lists); outside a numbered paragraph there is no subparagraph to name, and the closing text simply returns to the level of the provision. This also covers documents other than legislation: without it, the guidance that follows the conditions listed in TCPS 2 Article 3.7A would carry the label of its last condition. Listing 7 shows the labels the chunks of Article 101 receive.

Trade-offs. The rule relies on drafting conventions rather than on layout, so it works on text extracted from any source. It would close a list too early if its last item ran over several sentences with a line break after the first, which formal drafting rarely does. Articles whose lists are not followed by closing text, such as Article 7 of the AI Act, are labelled exactly as before.

6.3 Asking for a subdivision

The question is parsed for the subdivision it names (Table 6). A single letter is a point and two or more roman letters are a sub-point, so “point (i)” reads as a point and “point (iv)” as a sub-point. A number after *point* or *item* is read as a paragraph, because that is how people refer to numbered paragraphs in practice.

Written in the question	Read as
paragraph 2, para 2, subsection 2, 66(2)	paragraph 2
point d, letter d, (d)	point d
point (iv), subpoint iv, (iv)	sub-point iv
point 3, item 3	paragraph 3
second subparagraph, subparagraph 2	subparagraph 2

Table 6: Subdivisions recognised in a question.

Exact lookup (Section 3) fetches the whole provision, and the chunks that carry every requested label are then kept; a paragraph’s first subparagraph is its text without a subparagraph

label. If no chunk carries them all, the outer labels are dropped one at a time, so “66(2)(e)” still finds point (e) in an article whose points are not grouped into numbered paragraphs. Only the matching chunks are passed on: “paragraph 2 of Article 7” returns 14 chunks—the paragraph’s opening, points (a) to (k), and the two sub-points of point (k)—while “point (k) of paragraph 2 of Article 7” returns three, and “the second subparagraph of paragraph 1 of Article 101” returns the closing text alone. When the provision has no such subdivision, the lookup note lists what it has instead, which produces the deterministic reply described in Section 4.

6.4 Writing references for the reader

A passage’s labels are written back the way the reference would be cited: the provision, then the paragraph, point and sub-point in parentheses, then the subparagraph in words—“Article 66(2)(e)(iv)”, “Article 101(1), second subparagraph”—preceded by the printed page where the document has one. The same wording is used for the focus line given to the model (Section 4), so the model and the reader see the same reference.

7 Cross-reference graph

With passages labelled down to their subdivisions, the references between them can be resolved. The graph is built once per upload, stored beside the vectors, and read at answer time. This section follows it through its five stages: extracting references, turning them into edges, storing them, expanding the neighbourhood of an answer, and presenting it to the reader.

7.1 Extracting references

A reference is a unit word followed by an identifier and, optionally, a paragraph in parentheses, as in “Article 5(2)”. The extractor uses the same unit vocabulary as the structure parser, so anything that can be cited can also be a heading, and the reverse. A match that stands alone on its line is a heading, not a reference, and is skipped.

Each reference is then classified as internal or external. It points at another instrument when it is followed by “of” and a capitalised name that is not itself a unit (“Article 11 of Regulation (EU) 2019/2144”), or by an acronym of three or more capitals (“Article 114 TFEU”). “Article 5 of this Regulation” stays internal, because “this” is not a name. References are deduplicated per passage by unit, identifier and instrument. Listing 8 shows the result on a sample passage.

7.2 Building edges

A node is a provision: the most specific provision-level label a passage carries—recital, article, clause, rule, section, annex, schedule, appendix, exhibit, chapter, part or title—written as `kind:value`, for example `article:66`. The builder reads the document’s passages back from the vector store in reading order and, for each reference in each passage, adds an edge from the passage’s provision to the provision cited. An edge is *internal* when some passage of the document carries the cited label, *external* when it names another instrument, and *unresolved* when the document has no passage with that label. A provision’s references to itself are skipped.

1. As referred to in Article 5(2) and Annex III, the requirements of Chapter III, Section 2 apply. Article 11 of Regulation (EU) 2019/2144 applies, as does Article 114 TFEU. See also Article 5 of this Regulation.

article 5, paragraph 2	internal
annex III	internal
chapter III	internal
section 2	internal
article 11	external: Regulation (EU) 2019/2144
article 114	external: TFEU

Listing 8: References extracted from a sample passage.

```

1 CREATE TABLE IF NOT EXISTS edges (
2     position    INTEGER NOT NULL,
3     locator     TEXT NOT NULL,
4     document    TEXT NOT NULL,
5     source      TEXT NOT NULL,
6     target      TEXT NOT NULL,
7     edge_type   TEXT NOT NULL,
8     PRIMARY KEY (document, source, target, position)
9 );
10 CREATE INDEX IF NOT EXISTS edges_by_target ON edges (document, target);

```

Listing 9: The edge table (`graph/store.py`).

Every edge also records its *position*, the index of the citing passage, which orders a provision’s references as they appear in the text, and its *locator*, the paragraph, subparagraph, point and sub-point of the citing passage (“paragraph 1 · point b”).

Reading the passages back from the store, rather than from the parser’s output, guarantees that edges use exactly the labels retrieval uses: a node identifier converts directly into the metadata filter that fetches its text.

Trade-offs. Nodes are provisions, not paragraphs. References to “Article 5(2)” and “Article 5(3)” therefore reach the same node, and the precision on the cited side is traded for a graph that stays small and whose every node can be fetched with one filter. Precision on the citing side is kept in the locator. For the AI Act the graph holds 575 internal edges, 79 edges to other instruments and 29 unresolved ones, the latter mostly references to units the parser does not label as provisions, such as the sections inside an annex.

7.3 Storage

Edges are stored in SQLite (Listing 9). A document’s edges are replaced whenever it is uploaded and removed when it is deleted. Two queries serve every use: the edges leaving a set of nodes, ordered by position, and the edges arriving at a set of nodes, which the index on the target makes cheap.

Trade-offs. A graph database would add a server to install and run, for queries that are all one hop long. SQLite ships with Python, keeps the edges in a single file beside the vector store,

and answers both queries from an index. Traversals longer than one hop are done in Python with a bounded depth, which is all the interface needs.

7.4 Neighbourhood of an answer

Every answer carries the cross-reference neighbourhood of its passages. The provisions of the retrieved passages are the seeds, at depth 0. The neighbourhood is then expanded breadth-first along internal edges to depth 2, with at most 50 nodes. Each node carries its label, its kind (retrieved, internal, external or unresolved), its depth and, for provisions in the document, a preview of its first 240 characters. Each edge carries its type, its locator and an index that numbers a provision's references in reading order. Because the first level arrives with the answer, the references of a retrieved passage can be listed without another request.

Two endpoints serve deeper exploration on demand: one returns what a provision cites and what cites it, each in reading order, and the other returns a provision's full text (Table 5).

7.5 Presenting cross-references

A graph view, and why it was dropped. The first presentation was an interactive graph drawn beside the answer, first with a force-directed layout and then with a radial one that placed the retrieved provisions at the centre, arranged each further level on a wider ring, swapped nodes to shorten and uncross the arrows, and routed curved arrows around the boxes. It took about 1 500 lines of TypeScript. With the typical neighbourhood of 20 to 50 nodes it showed *that* provisions referred to each other, but not *what* they said: reading a chain meant opening node after node while holding the path in memory, and the layout's motion competed with the text. The graph view was therefore dropped in favour of a much simpler interface, and the graph itself remained in the backend as the data behind it.

The reference tree. Under each citation card, the provisions the passage cites are listed in the order the text cites them, each with the locator of the citing passage (Listing 10). Opening a row fetches the cited provision's text and its own references once and shows them indented beneath it, so a chain of citations stays on screen and reads from top to bottom. A reference to another instrument is marked *other document*, an unresolved one *not found*, and a provision that already appears higher on the same path *already above*; none of these can be opened, which also makes every chain finite even when provisions cite each other. A long list shows twelve rows and offers the rest on request. Nothing animates, and every row exposes its open or closed state to assistive technology.

Trade-offs. The tree gives up the overview of the whole neighbourhood that a graph offers at a glance, and with it the ability to see clusters and hubs. In exchange, the text of every cited provision is readable in place and in order, which is what following a cross-reference is for. The component is about 140 lines, a tenth of the graph view, and costs two small requests per opened row instead of a layout computation on every change.

```

Cites
v 1. Article 91 . paragraph 1 . point b
  Article 91 Power to request documentation and information
  1. The Commission may request the provider of the general-purpose
    AI model concerned to provide the documentation ...
v 1. Article 68 . paragraph 3
  Article 68 Scientific panel of independent experts
  1. The Commission shall, by means of an implementing act, ...
> 2. Article 93 . paragraph 1 . point c

```

Listing 10: The reference tree under Article 101(1), with two levels opened (v open, > closed).

Status and Outlook

8 Status and limitations

8.1 Testing and continuous integration

The unit tests run without a language model. The ingestion tests check the PDF, Word and text readers and the chunk sizes. The generation tests replace Ollama with a stub that records the prompt, and check that the answer and detail are taken from the model’s JSON reply, that a reply which is not JSON is kept as the answer, that token usage is reported, that the focus line appears exactly when a named provision was found, and that interleaving keeps the strongest passage first. A pipeline test ingests a PDF fixture and checks that retrieval returns passages with their locations; it needs the embedding model and the vector store and is run locally.

Continuous integration runs on GitHub Actions for every pull request and every push to the main branch. One job installs the interface’s dependencies, type-checks it and builds it; the other installs the backend on Python 3.12 with a CPU-only build of PyTorch and runs the eleven model-free tests.

8.2 Measured behaviour

Table 7 lists what ingestion recovers from the two main development documents, and Table 8 how representative questions are served. Times were measured on the development machine described in Section 1.

Document	Pages	Chunks	Structure found
AI Act [7]	144	2019	13 chapters, 5 sections, 113 articles, 180 recitals, 13 annexes; printed pages equal PDF pages
TCPS 2 (2022) [8]	287	1664	13 chapters, 142 articles; printed page = PDF page – 8

Table 7: Structure recovered at ingestion.

8.3 Limitations

- Structure is recognised from the numbering and punctuation conventions of formal drafting. A document that does not follow them is chunked without labels and searched by similarity

Question	Path	Passages	Retrieval
“paragraph 2 article 7” (AI Act)	exact, subdivision	14	3.1 ms
“chapter 3 page 48” (TCPS 2)	exact, chapter and page	7	5.5 ms
“paragraph 2 article 85” (AI Act)	exact, part missing	2	–
“who can lodge a complaint with a market surveillance authority” (AI Act)	hybrid	5	235 ms

Table 8: Retrieval for representative questions. For the missing paragraph the reply is composed without calling the model; for the hybrid question the provision on the right to lodge a complaint, Article 85, is ranked first. Generating an answer adds 7–9 s once the model is loaded.

only.

- A question is resolved to one provision at a time. A question relating two provisions (“what does paragraph 3 of this article have to do with Article 7”) retrieves only one of them.
- References to other instruments are recognised and marked, but not followed into other uploaded documents.
- The local 3B model and its 4096-token context window limit the passages to about 2000 words and keep summaries short; an amount or a condition may be left out of a plain-language summary.
- Printed page numbers are recovered only from PDF files with one sequential numbering; Word and text files have no pages, and scanned PDF files without a text layer are not read.
- The accuracy of answers has not yet been quantified on an annotated set of questions.

8.4 Future work

- Retrieve every provision a question names, resolve references such as “this article” from the conversation, and give the model the cross-reference path between the provisions, so that relational questions can be answered.
- Build an evaluation set of annotated questions and measure retrieval (whether the expected provision is among the passages) and faithfulness (whether every claim in the answer is supported by a cited passage).
- Add a verification step that checks each sentence of an answer against the passages it cites.
- Follow references to other instruments when those instruments are among the uploaded documents.

8.5 Summary

The system implements the RAG technique as a Python pipeline that answers questions grounded in uploaded documents and cites its sources. Every answer is drawn from retrieved passages, and every passage carries its document, printed page and structural reference. What distinguishes it from a generic pipeline is that it treats document structure as data. A question that names a

location is answered by exact lookup instead of similarity, returning exactly the part named—or saying that it does not exist. The number of passages follows from the question instead of a setting. And the cross-references between provisions are extracted into a graph that the reader can follow from any answer, through a deliberately simple reference tree.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [2] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [3] G. V. Cormack, C. L. A. Clarke and S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- [4] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- [5] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [7] Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). *Official Journal of the European Union*, L, 12.7.2024.
- [8] Canadian Institutes of Health Research, Natural Sciences and Engineering Research Council of Canada, and Social Sciences and Humanities Research Council of Canada. *Tri-Council Policy Statement: Ethical Conduct for Research Involving Humans – TCPS 2 (2022)*, 2022.